

COMPLETE EDITION

SOC ANALYST

1 • 2 • 3

THE COMPLETE FIELD GUIDE

50 chapters • worked examples • highlighted key terms

SecTechMaster



Foundation



Advanced Analyst



Senior /
Detection & Hunting



Cloud Security

REAL SKILLS. REAL KNOWLEDGE.

SOC ANALYST

1 • 2 • 3

THE COMPLETE FIELD GUIDE

SecTechMaster

SOC Analyst 1-2-3: The Complete Field Guide — Complete Edition

© 2026 SecTechMaster. All rights reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

All scenarios, companies, people, hostnames, IP addresses, domains and data in this book and in the companion app are fictional and created for training. IP addresses come from private and documentation ranges. Any resemblance to real organisations or events is coincidental.

MITRE ATT&CK® is a registered trademark of The MITRE Corporation. ATT&CK content © The MITRE Corporation, reproduced and distributed with the permission of The MITRE Corporation.

Microsoft, Azure, Microsoft Sentinel, Microsoft 365, Entra ID, Windows, AWS, Splunk, Wazuh, Kali Linux, Wireshark, Nmap, Suricata, Zeek and other product names are trademarks of their respective owners. SecTechMaster is not affiliated with or endorsed by them. Google Play and the Google Play logo are trademarks of Google LLC.

The techniques in this book are described so that defenders can recognise and stop them. Practise only in environments you own or are explicitly authorised to test. The information is provided “as is”, without warranty; the publisher is not liable for any loss arising from its use.

Published by SecTechMaster.

How to Use This Book

This guide is built to be studied with a highlighter — physical or digital — and practised with the **SOC Analyst 1-2-3 app** next to you. Every chapter follows the same shape, and the same boxes repeat throughout so you can scan, mark up and revise quickly:

KEY TERMS

Mint boxes — Key Terms

The vocabulary you need to know cold before an interview or a shift. Highlight these first.

WORKED EXAMPLE: Navy boxes

A realistic scenario walked through end to end, with fictional but plausible IPs, hostnames, log lines and queries — the closest thing to a real shift you will get from a book.

KEY TAKEAWAYS

- **Amber boxes** — the short version of each chapter: what to remember if you forget everything else.

COMMON PITFALLS

- **Red boxes** — mistakes real analysts make at this level. Read them twice.

► **PRACTICE THIS IN THE APP**

- **Blue boxes** — the exact screens in the app where you practise the chapter: the lesson and quiz, the lab, and the cases.

The book follows the **same order as the app**. It starts with networking, because almost every alert is a network event, then walks through the attacks you will meet on the lab network, and only then climbs the SOC career ladder from Level 1 to Level 3, finishing with cloud security on AWS and on Azure/Microsoft 365. Read it front to back if you are starting out, or jump to the part that matches where you are now.

The best way to study one chapter

1. Read the chapter and highlight the key terms.
2. In the app, open the matching lesson under **Learn** and pass its quiz.
3. Work the lab or case named in the blue box.
4. Come back to the takeaways and pitfalls, and check you can explain each one in your own words.

The SOC Career Progression

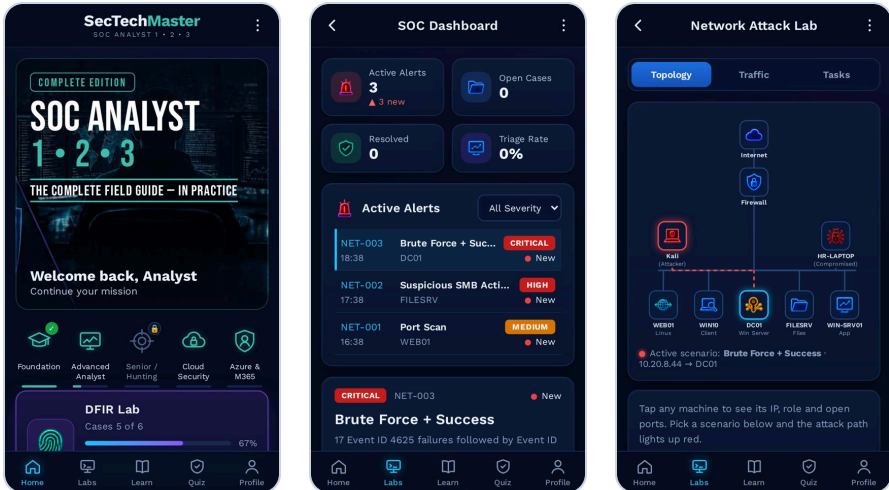
Part	Level	Goal	Progression model
I	Network Foundations	Understand how data moves and where you can see it	CONNECT → ADDRESS → CONTROL → OBSERVE
II	Network Attack Scenarios	Recognise attacks from the telemetry they leave	ATTACK → TELEMETRY → ALERT → DECIDE
III	SOC Level 1 — Foundation	Detect and investigate common security alerts	DETECT → TRIAGE → INVESTIGATE → ESCALATE
IV	SOC Level 2 — Advanced Analyst	Investigate complex attacks and hunt for threats	INVESTIGATE → CORRELATE → HUNT → SCOPE → RESPOND
V	SOC Level 3 — Senior / Detection & Hunting	Lead investigations and build better detections	HUNT → DETECT → ENGINEER → LEAD → AUTOMATE → IMPROVE
VI	Cloud Security (AWS)	Apply SOC skills to cloud identity, storage, network and detection	UNDERSTAND → SECURE → DETECT → HUNT
VII	Azure & Microsoft 365 Security	Apply SOC skills to Entra ID, Azure and Microsoft Sentinel	UNDERSTAND → SECURE → DETECT → HUNT

Notice how the verbs change as you move up. The network parts are about **seeing**: knowing what normal traffic looks like and what an attack leaves behind. Level 1 is about **recognising and escalating correctly**. Level 2 adds **correlation and proactive hunting**. Level 3 is no longer about one alert at a time — it is about **building the detections, systems and team habits** that make Levels 1 and 2 possible at scale.

The two cloud parts are not a higher tier of seniority. They apply the same Level 1-2-3 skills to different terrain, first AWS and then Azure and Microsoft 365, where the network is an API, the perimeter is identity, and the audit log is the closest thing you get to a packet capture.

Practice in the App

Every chapter in this book has a matching place to practise in the **SOC Analyst 1-2-3 app**. Reading tells you what an attack looks like; working it on a simulated network is what makes you fast and confident on a real shift.



What is in the app

- **Learn** — the chapters of this book as lessons, each with a quiz.
- **Labs** — hands-on labs grouped as Network, SOC/SIEM, Cloud, Investigation, and Engineering & Leadership, including the Network Attack Lab used throughout Parts I and II.
- **Cases** — fictional investigations across Levels 1, 2 and 3, from a single brute-force alert to a full attack chain.
- **SOC Dashboard** — a live alert queue with an investigation workflow that links each alert back to the chapter that teaches it: *Lesson* → *Alert* → *Investigate* → *Quiz* → *Close/Escalate*.
- **Profile** — XP, levels, achievements and your learning path, so you can see your progress through the book.

How the book and the app fit together

The app follows the same order as this book: Network Foundations, Network Attack Scenarios, then SOC Levels 1, 2 and 3, Cloud, and Azure & Microsoft 365. At the end of each chapter, the blue **Practice this in the app** box tells you exactly which lesson, lab and cases to open.

Get the app: search for *SOC Analyst 1-2-3* by SecTechMaster on Google Play.

PART I • NETWORK FOUNDATIONS

Goal: Understand how data moves across a network, and where a SOC analyst can see it.

CONNECT → ADDRESS → CONTROL → OBSERVE

Almost every alert a SOC analyst ever touches is, at its core, a network event. A host connected to something. A packet crossed a boundary. A name was looked up. A login came in from an address that should not be there. This part gives you the ground you will stand on for the rest of the book, and it follows exactly the same order as the SOC Analyst 1-2-3 app: networking first, then attacks, then the SOC levels.

You will learn how data is packaged and carried, how hosts are addressed, which ports and services matter, how the core services — DNS, DHCP and ARP — work and fail, which security controls sit in the path, and which logs they leave behind. The part ends with a tour of the lab network used throughout the app and the rest of this book, so every IP address and hostname you meet later already feels familiar.

What this part covers

1. How Networks Carry Data
2. IP Addressing, Subnets & NAT
3. TCP, UDP, Ports & Services
4. Core Network Services: DNS, DHCP & ARP
5. Network Security Controls
6. Network Telemetry: Where the Evidence Lives

7. Packet Analysis with Wireshark

8. The Lab Network: Assets, Segments & Normal Traffic

► **PRACTICE THIS IN THE APP**

In the app: Labs → **Network Labs** → Network Attack Lab (Topology and Traffic tabs), and Learn → Level 1 → Networking Fundamentals and Wireshark Basics.

1. How Networks Carry Data

You cannot investigate what you cannot describe. When an alert says “outbound connection on TCP/443” or “SMB session to FILESRV”, it is describing data that was packaged, addressed and carried across a network. This chapter gives you the mental model for that journey — the layers, the packaging and the path — so that every log line you read later tells a story instead of looking like noise.

KEY TERMS

Packet

A small unit of data with a header (who it is from, who it is for, how to handle it) and a payload (the data itself).

OSI model

A seven-layer reference model describing how data moves from an application down to the wire and back up again.

TCP/IP model

The practical four-layer model the internet actually runs on: Link, Internet, Transport and Application.

Encapsulation

Each layer wraps the data from the layer above in its own header before passing it down.

MAC address

The hardware address of a network interface, used to deliver frames on the local network (Layer 2).

IP address

The logical address of a host, used to route packets between networks (Layer 3).

Port

A number that identifies which service or conversation on a host the data belongs to (Layer 4).

The layers, and why an analyst cares

The **OSI model** splits networking into seven layers: Physical, Data Link, Network, Transport, Session, Presentation and Application. Few analysts recite all seven on shift, but the model gives you a shared vocabulary for *where* something is happening:

Layer	Name	What lives here	What you see in the SOC
7	Application	HTTP, DNS, SMB, RDP	URLs, queries, file names, user agents
4	Transport	TCP, UDP	Ports, flags, connection state
3	Network	IP, ICMP	Source and destination IP addresses
2	Data Link	Ethernet, Wi-Fi	MAC addresses, ARP
1	Physical	Cables, radio	Rarely — link up/down events

In day-to-day work you will think in the simpler **TCP/IP model**: Link (layers 1–2), Internet (layer 3), Transport (layer 4) and Application (layers 5–7). The practical rule is this: a firewall log with IPs and ports is showing you layers 3 and 4; a proxy log with a full URL has moved up to layer 7. Knowing which layer a log describes tells you what it *can* and *cannot* prove. A firewall log can prove two hosts talked on port 443; it cannot tell you which web page was requested.

Encapsulation: data in envelopes

When a browser sends a request, the data travels down the stack and each layer adds its own envelope:

1. The **application** creates the message — for example `GET /login HTTP/1.1`.
2. The **transport** layer adds a TCP header with source and destination ports (say 51322 → 443) and sequence numbers.
3. The **internet** layer adds an IP header with source and destination addresses (10.20.8.21 → 203.0.113.45).
4. The **link** layer adds an Ethernet frame with the MAC address of the next hop — usually the default gateway, not the final server.

The receiving side unwraps the envelopes in reverse order. This matters in investigations because different tools see different envelopes. A switch sees MAC addresses. A router and firewall see IP addresses and ports. A proxy, an IDS with TLS inspection, or the endpoint itself can see the application data. When you need to prove something, you go to the tool that sees the right envelope.

Local delivery versus routing

A host decides first whether the destination is on its **own subnet** or somewhere else. If it is local, the host uses ARP to find the destination's MAC address and delivers the frame directly. If it is remote, the host sends the frame to its **default gateway** (a router or firewall), which forwards the packet toward its destination hop by hop. You will learn subnets properly in Chapter 2, but hold on to this idea now: traffic that stays inside a subnet may never touch the firewall, which means the firewall log may never see it.

This is why SOC teams talk about two directions of traffic:

- **North–south** traffic crosses the perimeter — from inside to the internet or the reverse. It passes the edge firewall and proxy, so it is usually well logged.

- **East–west** traffic moves between internal hosts. It is where lateral movement happens, and it is often the *least* logged traffic in a network.

Following one connection end to end

Put the pieces together and every connection answers five questions — the **5-tuple**: source IP, source port, destination IP, destination port and protocol. Almost every network log you will ever read is built around it. If you can extract the 5-tuple, you can pivot: search the firewall for the same pair of IPs, search DNS for the name that resolved to the destination, search the endpoint for the process that owned the source port.

■ WORKED EXAMPLE: Reading one line at three layers

An IDS alert fires at 10:31:12 on the lab network: `TCP 10.20.8.44:44120 → 10.20.5.10:22 SYN`. You read it layer by layer. **Layer 3**: the source 10.20.8.44 is an internal address in the user segment; the destination 10.20.5.10 is WEB01 in the server segment, so this is **east–west** traffic crossing an internal boundary. **Layer 4**: the destination port is 22 (SSH) and only the SYN flag is set — a connection *attempt*, not an established session. Over the next four seconds the same source sends SYNs to ports 80, 443, 445, 3389 and 8080 on the same host. No single packet is alarming, but the pattern — one source, many ports, no completed handshakes — is the classic shape of a port scan. You now know *what* happened before you open a single dashboard, and you know the next pivot: which machine is 10.20.8.44, and who was logged on to it?

✓ KEY TAKEAWAYS

- Every log describes one or more layers; the layer tells you what the log can and cannot prove.
- Encapsulation means different tools see different envelopes — go to the tool that sees the envelope you need.
- Traffic inside a subnet may never reach the firewall; east-west traffic is where lateral movement hides.
- The 5-tuple (source IP, source port, destination IP, destination port, protocol) is the key you pivot on.

▲ COMMON PITFALLS

- Treating a firewall “allow” as proof of what the application did — it only proves that a connection was permitted.
- Assuming the MAC address in a capture belongs to the remote server; it is usually the gateway’s.
- Forgetting that internal-to-internal traffic may be invisible to the edge firewall.

► PRACTICE THIS IN THE APP

- **Learn:** Level 1 → Networking Fundamentals — read it, then pass the quiz.
- **Lab:** Labs → Network Labs → Network Attack Lab → **Topology** tab. Tap each machine and note its IP address and role.
- **Try it:** choose the Port Scan scenario and open the **Traffic** tab to watch the SYN packets from this chapter’s example.

2. IP Addressing, Subnets & NAT

An IP address is the first thing you look at in almost every alert, and it answers three questions faster than any tool: is this host inside or outside, which part of the network is it in, and is it really the host I think it is? This chapter teaches you to read addresses, subnets and NAT the way an analyst does — quickly, and with an eye on what they hide.

KEY TERMS

IPv4 address

A 32-bit address written as four numbers from 0 to 255, for example 10.20.1.5.

Private address (RFC 1918)

An address range reserved for internal networks and never routed on the internet: 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16.

Subnet

A group of addresses that can talk to each other directly, without going through a router.

CIDR notation

The “/number” after an address that says how many bits identify the network, for example /24 = 255.255.255.0.

Default gateway

The router or firewall a host sends traffic to when the destination is outside its own subnet.

NAT (Network Address Translation)

Rewriting addresses as traffic crosses a device — usually many private addresses sharing one public address on the way to the internet.

IPv6

The 128-bit successor to IPv4, written in hexadecimal groups such as `2001:db8::10`.

Reading an address in two seconds

When an IP address appears in an alert, ask first whether it is **private** or **public**. The private ranges — `10.0.0.0/8`, `172.16.0.0/12` and `192.168.0.0/16` — only exist inside organisations. If both ends of a connection are private, the traffic is internal (east–west). If one end is public, it crosses the perimeter (north–south) and the public side may belong to a cloud service, a partner, or an attacker.

A few other ranges are worth memorising because they mean something specific:

Range	Meaning	What it tells you in an alert
<code>127.0.0.0/8</code>	Loopback (the host itself)	The process talked to itself — often a local proxy or tool
<code>169.254.0.0/16</code>	Link-local (APIPA)	The host failed to get an address from DHCP
<code>100.64.0.0/10</code>	Carrier-grade NAT	Shared addresses at an ISP or VPN provider
<code>192.0.2.0/24</code> , <code>198.51.100.0/24</code> , <code>203.0.113.0/24</code>	Documentation ranges	Used in training material and examples, like this book
<code>224.0.0.0/4</code>	Multicast	Discovery traffic (mDNS, SSDP) — usually noise

Subnets and CIDR without the maths anxiety

A **subnet mask** splits an address into a network part and a host part. CIDR notation writes the mask as a count of network bits. The three you will see most:

- **/24** — 255.255.255.0 — 256 addresses (254 usable). 10.20.8.0/24 covers 10.20.8.1 to 10.20.8.254.
- **/16** — 255.255.0.0 — 65,536 addresses. 10.20.0.0/16 covers 10.20.0.0 to 10.20.255.255.
- **/32** — a single host. Firewall rules and threat intelligence often list /32 entries.

The analyst's use of subnets is practical, not academic: **are these two hosts in the same segment?** 10.20.8.21 and 10.20.8.44 are both in 10.20.8.0/24, the user segment, so they can reach each other without a firewall in between. 10.20.8.44 and 10.20.1.5 are in different /24s, so that traffic had to cross a router or internal firewall — and may have been logged there. Organisations usually give each subnet a purpose (users, servers, management, guest Wi-Fi), so the subnet alone tells you a lot about what a host *should* be doing.

The default gateway and routing

Every host has a routing table. For anything outside its own subnet, it sends traffic to the **default gateway**. In a simple network that is one firewall; in a larger one it is a chain of routers and firewalls. This is why a workstation's traffic to the internet and to a server in another subnet both leave through the same next hop — and why that next hop is such a valuable place to collect logs.

NAT: the address in the log may not be the host

NAT rewrites addresses in flight. The most common form, sometimes called PAT or “masquerade”, lets hundreds of internal hosts share one public IP. The firewall keeps a table: *internal 10.20.8.21:51322 is currently using public 198.51.100.7:40211*. Replies come back to the public port and the firewall rewrites them to the right internal host.

For an analyst, NAT creates two classic traps:

1. **Outbound:** a threat-intelligence report or an external partner says “your IP 198.51.100.7 attacked us”. That is your firewall’s public address, shared by everyone. To find the real host you must look up the **NAT translation log** for that exact time and source port.
2. **Inbound:** a port forward (destination NAT) sends traffic for public-IP:443 to an internal web server. Logs on the web server show the attacker’s public address; logs on the firewall show both. If a reverse proxy or load balancer sits in front, the server may only see the proxy’s address — check for headers such as `X-Forwarded-For`.

DHCP leases: the address may have changed hands

Most workstations get their address from **DHCP** (Chapter 4) for a limited lease time. The laptop that had 10.20.8.44 yesterday may not have it today. Before you name a host in an escalation, match the IP to a hostname **at the time of the event** using DHCP logs, EDR records or authentication logs — never from today’s inventory alone.

A word on IPv6

Many networks run IPv6 alongside IPv4, sometimes without the security team noticing. IPv6 addresses are long (`fe80::1c2d:3eff:fe4a:9b10`), hosts often have several at once, and

link-local addresses (fe80::/10) appear on every interface. The practical risks: detections and firewall rules written only for IPv4, and attacks that abuse IPv6 auto-configuration on networks that “don’t use IPv6”. If you see IPv6 in logs you did not expect, treat it as a visibility gap to report.

■ WORKED EXAMPLE: Finding the real host behind a shared public IP

A partner emails at 14:05: “Your address **198.51.100.7** sent 40 login attempts to our VPN portal between 13:10 and 13:14.” That address is your company’s shared outbound NAT address, so it identifies nothing on its own. You open the firewall’s NAT/session log and filter on translated source 198.51.100.7, destination = the partner’s VPN IP, time 13:10–13:14. Three internal sessions match, all from **10.20.8.44** with different source ports. DHCP shows 10.20.8.44 was leased to host **LAB-KALI** at that time — a machine that should only ever run approved red-team exercises. There is no exercise on the change calendar. You now have the true source, the time window and the reason it matters, and you escalate with the NAT log lines attached instead of guessing.

✓ KEY TAKEAWAYS

- Check private versus public first — it tells you east–west or north–south in a second.
- CIDR tells you whether two hosts share a segment, and therefore whether a firewall could have seen them.
- A public source IP in an outbound report is usually your NAT address — resolve it with NAT logs and the source port.
- Tie IPs to hostnames at the time of the event (DHCP, EDR, authentication logs), not from today’s inventory.
- Unexpected IPv6 is a visibility gap worth reporting.

▲ COMMON PITFALLS

- Blaming the edge firewall’s public address as if it were an attacking workstation.
- Reading today’s DHCP lease and naming the wrong laptop in an escalation.
- Missing a proxy or load balancer, so the “attacker IP” in the server log is really your own device.

► PRACTICE THIS IN THE APP

- **Learn:** Level 1 → Networking Fundamentals (the IP addressing and NAT sections), then its quiz.
- **Lab:** Network Attack Lab → **Topology** — each machine shows its IP address. Group them into subnets: which ones share 10.20.8.0/24?
- **Try it:** in the scenario controls, change the **Source IP** from 10.20.8.44 to 10.20.8.51 and watch which machine lights up as the attacker.

3. TCP, UDP, Ports & Services

If the IP address tells you *who* talked, the transport layer tells you *how* and *about what*. TCP flags reveal whether a connection really happened. Port numbers are the fastest clue to which service was involved. And the difference between TCP and UDP explains why some attacks are noisy and others are almost silent. This chapter turns those details into triage instincts.

KEY TERMS

TCP

Connection-oriented transport. It sets up a session with a handshake, numbers every byte and retransmits anything lost.

UDP

Connectionless transport. It sends datagrams with no handshake and no delivery guarantee — fast and simple.

Three-way handshake

SYN → SYN-ACK → ACK: the exchange that opens every TCP connection.

TCP flags

Bits in the TCP header that describe a packet's role: SYN (open), ACK (acknowledge), FIN (close politely), RST (abort), PSH, URG.

Port

A number from 0 to 65535 that identifies a service (server side) or a conversation (client side).

Well-known ports

Ports 0–1023, reserved for standard services such as 22/SSH, 53/DNS and 443/HTTPS.

Ephemeral port

A temporary high-numbered port the client picks for its side of a connection.

The TCP handshake and what each result means

Every TCP connection starts the same way. The client sends **SYN**. If a service is listening, the server answers **SYN-ACK**, and the client completes the handshake with **ACK**. Only then does data flow. At the end, each side sends **FIN** (a polite close) or one side sends **RST** (an abrupt reset).

This small exchange is one of your most useful pieces of evidence, because it tells you how far a connection got:

What you see	What it means	Typical cause
SYN, no reply	Nothing answered — filtered or host down	Firewall drop, scan against a closed network
SYN → RST	Host is up, port is closed	Scan hitting a closed port
SYN → SYN-ACK → RST (from client)	Port open, client aborted	“Half-open” SYN scan (for example Nmap -sS)
Full handshake, data, FIN	A real, completed session	Normal use — or a successful attack

When an alert says “connection attempt”, check which row you are in. Hundreds of SYNs with no completed sessions is reconnaissance. One completed session that moved data is a different conversation entirely.

UDP: quiet, fast and easy to fake

UDP has no handshake. A client sends a datagram and hopes. That makes it ideal for DNS lookups, time sync (NTP), streaming and VPN tunnels — and it also has consequences for security:

- There is no connection state to prove a two-way conversation, so a single UDP packet proves very little.
- The source address is easy to **spoof**, which is why UDP services are abused for reflection and amplification DDoS attacks.
- UDP scans are slow and ambiguous: silence can mean “open” or “filtered”.

A useful rule: for TCP, look for the handshake; for UDP, look for **request and response pairs** (a DNS query and its answer) and for **volume** (bytes and packets over time).

Ports: the fastest first clue

Knowing common ports by heart makes triage dramatically faster. A server listens on a known port; the client uses an ephemeral port (usually above 49152 on Windows, above 32768 on Linux). When you read a connection, the lower, well-known number is usually the service.

Port	Service	Why a SOC analyst cares
20/21 TCP	FTP	Clear-text credentials; old data transfer
22 TCP	SSH	Remote admin; brute-force target; tunnelling
23 TCP	Telnet	Clear-text remote access — should not exist
25/587 TCP	SMTP	Mail delivery; phishing path
53 UDP/TCP	DNS	Name lookups; C2 and tunnelling channel
67/68 UDP	DHCP	Address leases — who had which IP
80/443 TCP	HTTP/HTTPS	Web traffic; most malware C2 hides here
88 TCP/UDP	Kerberos	Domain authentication
135, 139, 445 TCP	RPC / NetBIOS / SMB	File shares; lateral movement; worms
389/636 TCP	LDAP/LDAPS	Directory queries; AD reconnaissance
1433/3306 TCP	MSSQL/MySQL	Databases — never exposed to the internet
3389 TCP	RDP	Remote desktop; brute-force and ransomware access
5985/5986 TCP	WinRM	Remote PowerShell; lateral movement

Ports are clues, not proof

A port number tells you what a connection *claimed* to be, not what it was. Attackers run SSH on 443 to slip through firewalls, and malware speaks custom protocols on port 80. Legitimate software also uses odd ports. So treat the port as a **hypothesis** and confirm it with deeper evidence: the process that owned the connection (EDR), the application data (proxy, IDS, packet capture) or the service banner.

■ WORKED EXAMPLE: Telling a scan from a session

Suricata raises “ET SCAN Possible Nmap SYN scan” for traffic from **10.20.8.44** to **WEB01 (10.20.5.10)**. You pull the flow records for the minute around the alert. The source sent SYNs to ports 22, 80, 443, 445, 3389 and 8080. WEB01 answered SYN-ACK on 22, 80 and 443 (open), RST on 445, 3389 and 8080 (closed), and the scanner replied RST to every SYN-ACK without finishing the handshake. There is **no completed session and no data transfer** — this is a half-open SYN scan that mapped three open services. The finding is real reconnaissance, but the impact so far is discovery only. Your note says exactly that, lists the three open ports as the attacker’s likely next targets, and asks the next shift to watch for authentication attempts on SSH and the web login page.

✓ KEY TAKEAWAYS

- The TCP handshake tells you how far a connection got — SYN only, reset, or a real session.
- UDP has no handshake; look for request/response pairs and volume, and remember sources can be spoofed.
- The lower, well-known port is usually the service; the high port is the client's ephemeral port.
- A port is a hypothesis about the service, not proof — confirm with process, application or packet evidence.

▲ COMMON PITFALLS

- Calling a scan a “compromise” when no handshake ever completed.
- Trusting that traffic on 443 must be normal web browsing.
- Reading the client's ephemeral port as the service being attacked.

► PRACTICE THIS IN THE APP

- **Learn:** Level 1 → Networking Fundamentals and Common Attacks (the reconnaissance section), with their quizzes.
- **Lab:** Network Attack Lab → choose **Port Scan**, execute it, then open the **Traffic** tab and match each line to the handshake table above.
- **Case:** Level 1 → Port Scanning / False Positive — decide whether the scan is a threat or approved activity.

End of the Free Sample

You have just read the opening of **SOC Analyst 1-2-3: The Complete Field Guide** — the networking foundation that everything else in a SOC is built on.

The complete book continues with 70 chapters in seven parts:

Part	What you will learn
I · Network Foundations	The rest of Part I: DNS, DHCP and ARP, security controls, network telemetry, Wireshark and the lab network
II · Network Attack Scenarios	Port scans, password attacks, RDP, SMB lateral movement, DNS abuse and the full credential attack chain — as they appear in the logs
III · SOC Level 1	SOC fundamentals, logs, SIEM, Wazuh, Splunk, Microsoft Sentinel, MITRE ATT&CK, triage, IOCs, documentation and incident response
IV · SOC Level 2	SPL and KQL, Sysmon, threat hunting, threat intelligence, detection engineering, malware, credential attacks, lateral movement, persistence, containment and DFIR
V · SOC Level 3	Advanced hunting, detection at scale, SIEM engineering, purple teaming, SOAR, detection-as-code, malware analysis, attack chains, SOC optimization and IR leadership
VI · Cloud Security — AWS	Shared responsibility, IAM, S3, VPC security, CloudTrail and GuardDuty, and cloud attack paths
VII · Azure & Microsoft 365	Entra ID, Storage and Key Vault, Azure networking, Microsoft Sentinel and hybrid identity attacks

Plus a 300-term glossary, Windows and Sysmon event ID reference, MITRE ATT&CK quick reference, career roadmap and app guide.

Every chapter comes with key terms, a worked example, takeaways, pitfalls and a **Practice this in the app** box pointing to the exact lesson, lab and cases.

► **PRACTICE THIS IN THE APP**

Get the full book as an ebook (EPUB and PDF) from SecTechMaster, and practise every chapter in the **SOC Analyst 1-2-3** app on Google Play.

© 2026 SecTechMaster. This sample may be shared freely in its complete, unmodified form.